Daniel Cuevas 300352904
Lab8

Task 1 Code:

```c
#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define REF_LEN 20

int main() {
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int n = REF_LEN;
    int num_frames = 3;
    int frames[MAX_FRAMES];
    int faults = 0;
    int next_replace = 0;

    memset(frames, -1, sizeof(frames));

    for (int i = 0; i < n; i++) {
        int page = ref[i];

        int found = 0;
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                found = 1;
                break;
            }
        }

        if (found) {
            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) printf("-");
                else printf("%d", frames[j]);
                if (j < num_frames - 1) printf(", ");
            }
            printf("] HIT\n");
        } else {
            faults++;
            int replaced = frames[next_replace];
            frames[next_replace] = page;
            next_replace = (next_replace + 1) % num_frames;

            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) printf("-");
                else printf("%d", frames[j]);
                if (j < num_frames - 1) printf(", ");
            }
            if (replaced == -1)
                printf("] FAULT\n");
            else
                printf("] FAULT (replaced %d)\n", replaced);
        }
    }

    printf("\nTotal Page Faults: %d\n", faults);
    return 0;
}
```

Output:

```
dcuev@iFHOX:~/cosc315/lab8$ ./fifo
Access 7: [7, -, -] FAULT
Access 0: [7, 0, -] FAULT
Access 1: [7, 0, 1] FAULT
Access 2: [2, 0, 1] FAULT (replaced 7)
Access 0: [2, 0, 1] HIT
Access 3: [2, 3, 1] FAULT (replaced 0)
Access 0: [2, 3, 0] FAULT (replaced 1)
Access 4: [4, 3, 0] FAULT (replaced 2)
Access 2: [4, 2, 0] FAULT (replaced 3)
Access 3: [4, 2, 3] FAULT (replaced 0)
Access 0: [0, 2, 3] FAULT (replaced 4)
Access 3: [0, 2, 3] HIT
Access 2: [0, 2, 3] HIT
Access 1: [0, 1, 3] FAULT (replaced 2)
Access 2: [0, 1, 2] FAULT (replaced 3)
Access 0: [0, 1, 2] HIT
Access 1: [0, 1, 2] HIT
Access 7: [7, 1, 2] FAULT (replaced 0)
Access 0: [7, 0, 2] FAULT (replaced 1)
Access 1: [7, 0, 1] FAULT (replaced 2)

Total Page Faults: 15
dcuev@iFHOX:~/cosc315/lab8$
```

Task 2 Code:

```c
#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define REF_LEN 20

typedef struct {
    int page;
    int last_used;
} Frame;

int main() {
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int n = REF_LEN;
    int num_frames = 3;
    Frame frames[MAX_FRAMES];
    int faults = 0;

    for (int i = 0; i < num_frames; i++) {
        frames[i].page = -1;
        frames[i].last_used = -1;
    }

    for (int i = 0; i < n; i++) {
        int page = ref[i];
        int found = 0;


        for (int j = 0; j < num_frames; j++) {
            if (frames[j].page == page) {
                found = 1;
                frames[j].last_used = i;
                break;
            }
        }

        if (!found) {
            faults++;
             int victim = 0;
            int empty = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j].page == -1) {
                    empty = j;
                    break;
                }
            }

            if (empty != -1) {
                victim = empty;
            } else {
                int min_time = frames[0].last_used;
                victim = 0;
                for (int j = 1; j < num_frames; j++) {
                    if (frames[j].last_used < min_time) {
                        min_time = frames[j].last_used;
                        victim = j;
                    }
                }
            }

            int replaced = frames[victim].page;
            frames[victim].page = page;
            frames[victim].last_used = i;

            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j].page == -1) printf("-");
                else printf("%d", frames[j].page);
                if (j < num_frames - 1) printf(", ");
            }
            if (replaced == -1)
                printf("] FAULT\n");
            else
                printf("] FAULT (replaced %d)\n", replaced);
        } else {
            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j].page == -1) printf("-");
                else printf("%d", frames[j].page);
                if (j < num_frames - 1) printf(", ");
            }
            printf("] HIT\n");
        }
    }

    printf("\nTotal Page Faults: %d\n", faults);
    return 0;
}
-- INSERT --
```

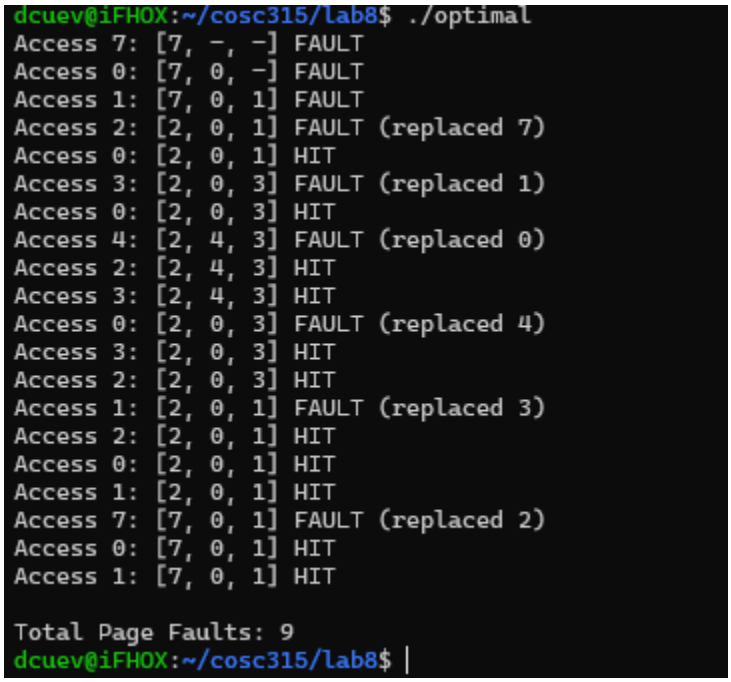
Output:

```
dcuev@iFHOX:~/cosc315/lab8$ ./lru
Access 7: [7, -, -] FAULT
Access 0: [7, 0, -] FAULT
Access 1: [7, 0, 1] FAULT
Access 2: [2, 0, 1] FAULT (replaced 7)
Access 0: [2, 0, 1] HIT
Access 3: [2, 0, 3] FAULT (replaced 1)
Access 0: [2, 0, 3] HIT
Access 4: [4, 0, 3] FAULT (replaced 2)
Access 2: [4, 0, 2] FAULT (replaced 3)
Access 3: [4, 3, 2] FAULT (replaced 0)
Access 0: [0, 3, 2] FAULT (replaced 4)
Access 3: [0, 3, 2] HIT
Access 2: [0, 3, 2] HIT
Access 1: [1, 3, 2] FAULT (replaced 0)
Access 2: [1, 3, 2] HIT
Access 0: [1, 0, 2] FAULT (replaced 3)
Access 1: [1, 0, 2] HIT
Access 7: [1, 0, 7] FAULT (replaced 2)
Access 0: [1, 0, 7] HIT
Access 1: [1, 0, 7] HIT

Total Page Faults: 12
dcuev@iFHOX:~/cosc315/lab8$
```

Task 3 Code:

```c
#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define REF_LEN 20

int main() {
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int n = REF_LEN;
    int num_frames = 3;
    int frames[MAX_FRAMES];
    int faults = 0;

    memset(frames, -1, sizeof(frames));

    for (int i = 0; i < n; i++) {
        int page = ref[i];
        int found = 0;

        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                found = 1;
                break;
            }
        }

        if (!found) {
            faults++;

            int empty = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    empty = j;
                    break;
                }
            }

            int victim;
            int replaced;
            if (empty != -1) {
                victim = empty;
            } else {

                int farthest = -1;
                victim = 0;
                for (int j = 0; j < num_frames; j++) {
                    int next_use = -1;
                    for (int k = i + 1; k < n; k++) {
                        if (ref[k] == frames[j]) {
                            next_use = k;
                            break;
                        }
                    }
                    if (next_use == -1) {

                        victim = j;
                        break;
                    }
                    if (next_use > farthest) {
                        farthest = next_use;
                        victim = j;
                    }
                }
            }

            replaced = frames[victim];
            frames[victim] = page;

            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) printf("-");
                else printf("%d", frames[j]);
                if (j < num_frames - 1) printf(", ");
            }
            if (replaced == -1)
                printf("] FAULT\n");
            else
                printf("] FAULT (replaced %d)\n", replaced);
        } else {
            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) printf("-");
                else printf("%d", frames[j]);
                if (j < num_frames - 1) printf(", ");
            }
            printf("] HIT\n");
        }
    }

    printf("\nTotal Page Faults: %d\n", faults);
    return 0;
}
```

Output:




Task 4 Code:

```c
#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define REF_LEN 20

int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
int n = REF_LEN;

int fifo(int num_frames) {
    int frames[MAX_FRAMES];
    memset(frames, -1, sizeof(frames));
    int faults = 0, next = 0;

    for (int i = 0; i < n; i++) {
        int page = ref[i], found = 0;
        for (int j = 0; j < num_frames; j++)
            if (frames[j] == page) { found = 1; break; }
        if (!found) {
            faults++;
            frames[next] = page;
            next = (next + 1) % num_frames;
        }
    }
    return faults;
}

int lru(int num_frames) {
    int pages[MAX_FRAMES], last[MAX_FRAMES];
    memset(pages, -1, sizeof(pages));
    memset(last, -1, sizeof(last));
    int faults = 0;

    for (int i = 0; i < n; i++) {
        int page = ref[i], found = 0;
        for (int j = 0; j < num_frames; j++) {
            if (pages[j] == page) {
                found = 1;
                last[j] = i;
                break;
            }
        }
        if (!found) {
            faults++;
            int victim = 0, empty = -1;
            for (int j = 0; j < num_frames; j++) {
                if (pages[j] == -1) { empty = j; break; }
            }
            if (empty != -1) {
                victim = empty;
            } else {
                int min_t = last[0]; victim = 0;
                for (int j = 1; j < num_frames; j++)
                    if (last[j] < min_t) { min_t = last[j]; victim = j; }
            }
            pages[victim] = page;
            last[victim] = i;
        }
    }
    return faults;
}

int optimal(int num_frames) {
    int frames[MAX_FRAMES];
    memset(frames, -1, sizeof(frames));
    int faults = 0;

    for (int i = 0; i < n; i++) {
        int page = ref[i], found = 0;
        for (int j = 0; j < num_frames; j++)
            if (frames[j] == page) { found = 1; break; }
        if (!found) {
            faults++;
            int empty = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) { empty = j; break; }
            }
            if (empty != -1) {
                frames[empty] = page;
            } else {
                int victim = 0, farthest = -1;
                for (int j = 0; j < num_frames; j++) {
                    int next_use = -1;
                    for (int k = i + 1; k < n; k++) {
                        if (ref[k] == frames[j]) { next_use = k; break; }
                    }
                    if (next_use == -1) { victim = j; break; }
                    if (next_use > farthest) { farthest = next_use; victim = j; }
                }
                frames[victim] = page;
            }
        }
    }
    return faults;
}
```

```
    return faults;
}

int main() {
    printf("Page Replacement Algorithm Comparison\n");
    printf("Reference String: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1\n\n");
    printf("%-8s %-8s %-8s %-8s %-10s\n", "Frames", "FIFO", "LRU", "OPT", "Best");
    printf("-----------------------------------------------\n");

    for (int f = 1; f <= 7; f++) {
        int fi = fifo(f);
        int lr = lru(f);
        int op = optimal(f);

        const char *best;
        if (op <= lr && op <= fi) {
            if (lr == op && fi == op) best = "ALL";
            else if (lr == op) best = "LRU/OPT";
            else best = "OPT";
        } else if (lr <= fi) {
            best = "LRU";
        } else {
            best = "FIFO";
        }

        printf("%-8d %-8d %-8d %-8d %-10s\n", f, fi, lr, op, best);
    }

    return 0;
}
-- INSERT --
```

Output:

```
dcuev@iFHOX:~/cosc315/lab8$ ./compare
Page Replacement Algorithm Comparison
Reference String: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

Frames   FIFO     LRU      OPT      Best
-----------------------------------------------
1        20       20       20       ALL
2        15       17       13       OPT
3        15       12       9        OPT
4        10       8        8        LRU/OPT
5        9        7        7        LRU/OPT
6        6        6        6        ALL
7        6        6        6        ALL
dcuev@iFHOX:~/cosc315/lab8$
```

Task 5:

Code:

```c
#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10

int belady_ref[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
int belady_n = 12;

int fifo(int *ref, int n, int num_frames, int verbose) {
    int frames[MAX_FRAMES];
    memset(frames, -1, sizeof(frames));
    int faults = 0, next = 0;

    for (int i = 0; i < n; i++) {
        int page = ref[i], found = 0;
        for (int j = 0; j < num_frames; j++)
            if (frames[j] == page) { found = 1; break; }
        if (!found) {
            faults++;
            int replaced = frames[next];
            frames[next] = page;
            next = (next + 1) % num_frames;
            if (verbose) {
                printf("Access %d: [", page);
                for (int j = 0; j < num_frames; j++) {
                    if (frames[j] == -1) printf("-");
                    else printf("%d", frames[j]);
                    if (j < num_frames - 1) printf(", ");
                }
                if (replaced == -1) printf("] FAULT\n");
                else printf("] FAULT (replaced %d)\n", replaced);
            }
        } else if (verbose) {
            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) printf("-");
                else printf("%d", frames[j]);
                if (j < num_frames - 1) printf(", ");
            }
            printf("] HIT\n");
        }
    }
    return faults;
}

int lru(int *ref, int n, int num_frames, int verbose) {
    int pages[MAX_FRAMES], last[MAX_FRAMES];
    memset(pages, -1, sizeof(pages));
    memset(last, -1, sizeof(last));
    int faults = 0;

    for (int i = 0; i < n; i++) {
        int page = ref[i], found = 0;
        for (int j = 0; j < num_frames; j++) {
            if (pages[j] == page) {
                found = 1;
                last[j] = i;
                break;
            }
        }
        if (!found) {
            faults++;
            int victim = 0, empty = -1;
            for (int j = 0; j < num_frames; j++) {
                if (pages[j] == -1) { empty = j; break; }
            }
            if (empty != -1) victim = empty;
            else {
                int min_t = last[0]; victim = 0;
                for (int j = 1; j < num_frames; j++)
                    if (last[j] < min_t) { min_t = last[j]; victim = j; }
            }
            int replaced = pages[victim];
            pages[victim] = page;
            last[victim] = i;
            if (verbose) {
                printf("Access %d: [", page);
                for (int j = 0; j < num_frames; j++) {
                    if (pages[j] == -1) printf("-");
                    else printf("%d", pages[j]);
                    if (j < num_frames - 1) printf(", ");
                }
                if (replaced == -1) printf("] FAULT\n");
                else printf("] FAULT (replaced %d)\n", replaced);
            }
        } else if (verbose) {
            printf("Access %d: [", page);
            for (int j = 0; j < num_frames; j++) {
                if (pages[j] == -1) printf("-");
                else printf("%d", pages[j]);
                if (j < num_frames - 1) printf(", ");
            }
            printf("] HIT\n");
        }
    }
    return faults;
}
```

```
    return faults;
}

int main() {
    printf("=== Belady's Anomaly Demonstration ===\n");
    printf("Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5\n\n");

    printf("--- FIFO with 3 frames ---\n");
    int f3 = fifo(belady_ref, belady_n, 3, 1);
    printf("Total Page Faults: %d\n\n", f3);

    printf("--- FIFO with 4 frames ---\n");
    int f4 = fifo(belady_ref, belady_n, 4, 1);
    printf("Total Page Faults: %d\n\n", f4);

    if (f4 > f3)
        printf("*** BELADY'S ANOMALY: FIFO with 4 frames (%d faults) > 3 frames (%d faults)! ***\n\n", f4, f3);

    printf("--- LRU with 3 frames ---\n");
    int l3 = lru(belady_ref, belady_n, 3, 1);
    printf("Total Page Faults: %d\n\n", l3);

    printf("--- LRU with 4 frames ---\n");
    int l4 = lru(belady_ref, belady_n, 4, 1);
    printf("Total Page Faults: %d\n\n", l4);

    if (l4 <= l3)
        printf("LRU does NOT exhibit Belady's anomaly: 4 frames (%d faults) <= 3 frames (%d faults)\n", l4, l3);

    return 0;
}
-- INSERT --
```

Output:

```
dcuev@iFHOX:~/cosc315/lab8$ ./belady
=== Belady's Anomaly Demonstration ===
Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

--- FIFO with 3 frames ---
Access 1: [1, -, -] FAULT
Access 2: [1, 2, -] FAULT
Access 3: [1, 2, 3] FAULT
Access 4: [4, 2, 3] FAULT (replaced 1)
Access 1: [4, 1, 3] FAULT (replaced 2)
Access 2: [4, 1, 2] FAULT (replaced 3)
Access 5: [5, 1, 2] FAULT (replaced 4)
Access 1: [5, 1, 2] HIT
Access 2: [5, 1, 2] HIT
Access 3: [5, 3, 2] FAULT (replaced 1)
Access 4: [5, 3, 4] FAULT (replaced 2)
Access 5: [5, 3, 4] HIT
Total Page Faults: 9

--- FIFO with 4 frames ---
Access 1: [1, -, -, -] FAULT
Access 2: [1, 2, -, -] FAULT
Access 3: [1, 2, 3, -] FAULT
Access 4: [1, 2, 3, 4] FAULT
Access 1: [1, 2, 3, 4] HIT
Access 2: [1, 2, 3, 4] HIT
Access 5: [5, 2, 3, 4] FAULT (replaced 1)
Access 1: [5, 1, 3, 4] FAULT (replaced 2)
Access 2: [5, 1, 2, 4] FAULT (replaced 3)
Access 3: [5, 1, 2, 3] FAULT (replaced 4)
Access 4: [4, 1, 2, 3] FAULT (replaced 5)
Access 5: [4, 5, 2, 3] FAULT (replaced 1)
Total Page Faults: 10

*** BELADY'S ANOMALY: FIFO with 4 frames (10 faults) > 3 frames (9 faults)! ***

--- LRU with 3 frames ---
Access 1: [1, -, -] FAULT
Access 2: [1, 2, -] FAULT
Access 3: [1, 2, 3] FAULT
Access 4: [4, 2, 3] FAULT (replaced 1)
Access 1: [4, 1, 3] FAULT (replaced 2)
Access 2: [4, 1, 2] FAULT (replaced 3)
Access 5: [5, 1, 2] FAULT (replaced 4)
Access 1: [5, 1, 2] HIT
Access 2: [5, 1, 2] HIT
Access 3: [3, 1, 2] FAULT (replaced 5)
Access 4: [3, 4, 2] FAULT (replaced 1)
Access 5: [3, 4, 5] FAULT (replaced 2)
Total Page Faults: 10

--- LRU with 4 frames ---
Access 1: [1, -, -, -] FAULT
Access 2: [1, 2, -, -] FAULT
Access 3: [1, 2, 3, -] FAULT
Access 4: [1, 2, 3, 4] FAULT
Access 1: [1, 2, 3, 4] HIT
Access 2: [1, 2, 3, 4] HIT
Access 5: [1, 2, 5, 4] FAULT (replaced 3)
Access 1: [1, 2, 5, 4] HIT
Access 2: [1, 2, 5, 4] HIT
Access 3: [1, 2, 5, 3] FAULT (replaced 4)
Access 4: [1, 2, 4, 3] FAULT (replaced 5)
Access 5: [5, 2, 4, 3] FAULT (replaced 1)
Total Page Faults: 8

LRU does NOT exhibit Belady's anomaly: 4 frames (8 faults) <= 3 frames (10 faults)
dcuev@iFHOX:~/cosc315/lab8$
```

Questions:

Q1: Why is the Optimal algorithm not used in real operating systems? What makes it impractical?

A1: Optimal requires knowledge of future page references to see what page is not going to be used for the longest time. The OS cannot predict which pages a process will access in the future, impossible to implement. It's only theoretical.

Q2: Explain why LRU generally performs better than FIFO. What assumption about program behavior does LRU exploit?
A2: LRU keeps in mind that pages that have been accessed recently will be accessed again in the future. FIFO only tracks insertion order and may take a page out just because it was loaded earliest.
Q3: What is Bélády's anomaly and why does it only affect FIFO but not LRU?
A3: Belady means that although it shouldn't, increasing the number of frames causes more page faults rather than fewer. FIFO is not a stack algorithm. LRU on the other hand, it is one, pages kept in memory are always a subset of those with more frames.
Q4: In practice, how might an OS approximate LRU without tracking exact access times for every page?
A4: OS uses the clock algorithm usually, which uses a reference bit for each page. The hardware sets its reference bit to 1 when accessed. When a victim is needed, the algorithm scans pages in circular order. This approximates LRU with very low overhead since the hardware handles setting the reference bit automatically.